

Streaming communication for scalable data exchange in DYNAMOS

Ruben Stoop

30th March 2026

Project Details

Project Title:	Streaming communication for scalable data exchange in DYNAMOS
Student:	Ruben Stoop
Academic Supervisors:	dr. Ana Oprescu, a.m.oprescu@uva.nl
Host:	Complex Cyber-Infrastructure (CCI), University of Amsterdam (https://cci-research.nl)
Contact Person:	Alexandros Koufakis, a.m.koufakis@uva.nl
Start Date:	1 April 2026
Planned End Date:	31 July 2026

1 Introduction

Many contemporary distributed systems operate in environments where data and computational resources are owned by different organizations. In such settings, data exchange must adhere to legal agreements, privacy regulations, and organizational policies. These constraints are common in domains such as healthcare, scientific research, and public-sector infrastructures, where unrestricted data exchange is not acceptable.

DYNAMOS (Dynamically Adaptive Microservice-based Operating System) is a data exchange middleware developed by the Complex Cyber-Infrastructure (CCI) group at the University of Amsterdam. The system implements a set of recurring data sharing archetypes that describe how data and computation are distributed among participants under policy constraints. DYNAMOS realizes these archetypes by dynamically composing chains of microservices that execute data sharing requests in compliance with formally evaluated policies. The middleware is designed to be adaptive, allowing architectural and execution decisions to change at runtime in response to policy requirements.

The current implementation of DYNAMOS uses gRPC for inter-service communication and relies exclusively on atomic request–response interactions. This communication

model is suitable for small or bounded data transfers but becomes limiting when applied to large-scale or continuous data exchange. In practice, data in distributed systems is often produced and consumed incrementally, for example in sensor data streams, distributed analytics pipelines, or staged processing of large research datasets.

When request-response communication is used in such scenarios, systems must either buffer large volumes of data before transmission or split transfers into multiple independent requests. Buffering increases memory usage and can lead to idle time between producers and consumers. Repeated request-response interactions introduce additional coordination overhead. Both effects negatively impact scalability and resource utilization as data volumes and the number of participating services increase.

Streaming-based communication has been proposed as a solution to these limitations. Streaming allows data to be transferred and processed incrementally, enabling overlap between computation and communication and reducing peak memory usage. gRPC supports several streaming interaction patterns, including client-side, server-side, and bidirectional streaming.

Integrating streaming communication into DYNAMOS is not straightforward. Streaming relies on long-lived connections and continuous data flows, which must be reconciled with DYNAMOS' policy-driven execution model, dynamic microservice composition, orchestration decisions, and fault handling mechanisms. Without careful design, the introduction of streaming may interfere with policy enforcement or reduce the ability to evaluate and reason about system behavior.

The goal of this project is to investigate how streaming communication can be incorporated into DYNAMOS in a way that improves scalability and performance while preserving policy compliance and adaptive behavior. This involves analyzing existing streaming communication mechanisms, designing architectural extensions to the middleware, implementing streaming support, and evaluating the resulting system under realistic workloads.

The main research question addressed in this thesis is:

Main RQ: How can streaming communication be integrated into a dynamically adaptive, policy-driven microservice-based data exchange middleware such as DYNAMOS, while preserving compliance guarantees and improving scalability and performance?

To answer this question, the following sub-research questions are defined:

- **RQ1:** Which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS?
- **RQ2:** How can streaming communication be integrated into a microservice-based architecture in a way that preserves dynamic service composition, policy-driven orchestration, and isolation guarantees?
- **RQ3:** What is the impact of streaming communication on scalability, performance, and robustness compared to a traditional request-response approach under realistic workloads in a microservice-based architecture?

2 Related Work

This section reviews existing literature related to streaming communication in microservice-based systems, with a particular focus on topics that are central to this thesis: streaming communication technologies and interaction models, architectural integration in adaptive microservice middleware, policy enforcement under continuous data exchange, resilience mechanisms for streaming workflows, and validation practices with respect to scalability and performance.

2.1 Streaming communication technologies and interaction models

DYNAMOS is implemented as a dynamically composed microservice system deployed on Kubernetes, using gRPC for inter-service communication and RabbitMQ for asynchronous messaging [20, 21]. This constrains the design space for introducing streaming functionality to a limited set of technologies that are compatible with containerized microservices and cloud-native deployment models. The primary question addressed in this body of work is which streaming communication mechanisms are suitable for large-scale or continuous data exchange, and under which conditions direct streaming RPC should be preferred over broker-based approaches.

gRPC provides native support for streaming through client-streaming, server-streaming, and bidirectional streaming RPCs. These interaction patterns differ in message flow control, lifetime of connections, buffering behaviour, and error propagation semantics. In the context of DYNAMOS, the choice of streaming pattern is not merely an API concern but has architectural consequences, as it affects memory usage, cancellation behaviour, and the ability to overlap communication and computation. The original DYNAMOS thesis identifies client-streaming as a promising mechanism for transferring large datasets incrementally, reducing peak memory pressure at the cost of increased handling complexity [20].

However, the use of long-lived streaming connections introduces operational challenges that are largely absent in unary request-response interactions. Starck and Ghofrani show that gRPC streaming complicates load balancing in Kubernetes environments, as persistent connections can cause traffic to become unevenly distributed across replicas [19]. This observation is particularly relevant for systems such as DYNAMOS that rely on horizontal scaling and ephemeral execution units. Complementary benchmarking work demonstrates that the performance characteristics of streaming gRPC are sensitive to runtime configuration and virtualization overhead, with measurable effects on latency and resource utilisation in containerized environments [18].

An alternative to direct service-to-service streaming is to externalise the data plane to a messaging or event streaming platform. Broker-based communication decouples producers and consumers in time, delegates buffering and persistence to the broker, and can simplify fan-out scenarios. Comparative benchmarks of Kafka, RabbitMQ, NATS, and ActiveMQ Artemis show that different brokers dominate under different workload characteristics, reinforcing that broker choice is highly workload dependent [8]. A cloud-native literature review comparing Kafka, Pulsar, and RabbitMQ further highlights trade-offs between throughput, reliability, operational complexity, and Kubernetes

integration [14]. While these studies are not conducted in the context of policy-driven middleware, they provide structured criteria that are directly applicable when evaluating broker-based alternatives for DYNAMOS.

Several studies also emphasize that streaming design is influenced not only by technology choice but by the underlying communication model. DataXc contrasts push-based streaming approaches with pull-based designs and reports improved latency stability and throughput under consumer-controlled flow [1]. Although the application domain differs, the distinction between producer-driven and consumer-driven streaming is directly relevant to backpressure management and resource utilisation in DYNAMOS-compatible workflows.

Overall, existing literature provides a broad comparison of streaming technologies and interaction patterns, but these comparisons are typically conducted in generic microservice or stream-processing settings. There is limited work that evaluates these mechanisms under the specific constraints of dynamically composed, policy-driven middleware with ephemeral execution units, leaving open questions regarding their suitability in such environments.

2.2 Architectural integration of streaming in adaptive microservice middleware

Beyond the selection of a streaming mechanism, integrating streaming communication into an adaptive microservice architecture raises architectural questions related to modularity, orchestration, and lifecycle management. DYNAMOS is explicitly designed around dynamic microservice composition, where execution chains are generated and deployed per request based on policy evaluation and system state [20, 21]. These chains are short-lived by design, which improves isolation and compliance guarantees but complicates the management of long-lived communication channels.

Communication in DYNAMOS is abstracted through a sidecar-based architecture, decoupling communication concerns from core microservice logic. The sidecar pattern is widely used in cloud-native systems to address cross-cutting concerns such as observability, security, and traffic management. Figueira and Coutinho demonstrate how sidecars can be integrated into self-adaptive microservice architectures on Kubernetes, enabling runtime adaptation without violating service modularity [2]. While their work does not explicitly address streaming data transfer, it illustrates how communication behaviour can be adapted dynamically at runtime.

Several middleware frameworks operationalise sidecars as first-class runtime components. Dapr provides a protocol-agnostic communication abstraction that supports service invocation, pub/sub, and state management through a sidecar runtime [2]. Dapr demonstrates that heterogeneous communication paradigms, including streaming and messaging, can be unified behind a stable interface. However, Dapr assumes relatively static service topologies and does not address dynamic chain composition driven by policy evaluation, limiting its applicability as a direct solution for DYNAMOS.

Existing architectural work therefore supports the feasibility of integrating streaming into sidecar-based microservice systems but does not address the combination of long-lived streaming connections with ephemeral, dynamically composed execution chains.

This limitation is particularly relevant in systems that combine adaptivity with strict policy enforcement requirements.

2.3 Policy enforcement and compliance under continuous data exchange

Policy enforcement is a defining characteristic of DYNAMOS, where access rights, execution locations, and data-flow constraints are evaluated prior to execution and enforced through dynamically composed microservice chains [20, 21]. In the current architecture, policy enforcement is implicitly scoped to bounded request–response interactions. Introducing streaming communication challenges this assumption, as data transfer may span longer periods during which policies, system state, or execution context may change.

Neumaier et al. propose a policy-aware architecture for decentralized dataset exchange that emphasizes continuous monitoring and runtime validation against machine-readable policies [13]. While their work does not explicitly target streaming RPC, it introduces an important distinction between one-time policy clearance and ongoing compliance monitoring, which becomes critical when data is exchanged incrementally over long-lived connections.

At the communication layer, several studies propose enforcing security and access control through proxies or sidecars. Thiagarajan et al. present a proxy-based approach for securing gRPC communication using mTLS, JWT authentication, and RBAC [22]. Although the focus is on security rather than data usage policy, the architectural approach is relevant, as it suggests that enforcement can be applied at communication boundaries and potentially at message granularity.

Despite these contributions, there is limited guidance on how policy enforcement should interact with long-lived streaming connections in systems that dynamically compose execution chains. In particular, existing work does not address how policy violations should be detected or handled mid-stream, or how streams should be terminated or adapted when compliance conditions change. These limitations highlight open challenges when introducing streaming into policy-driven middleware such as DYNAMOS.

2.4 Resilience, backpressure, and fault handling in streaming workflows

Streaming-based communication introduces operational challenges that are less pronounced in request–response systems. Long-lived data flows must tolerate partial failures, adapt to fluctuating load, and prevent uncontrolled resource growth through effective backpressure mechanisms. These challenges are compounded in DYNAMOS by dynamic microservice composition and ephemeral execution units.

Reactive microservice architectures emphasize asynchronous communication, non-blocking execution, and fault isolation as mechanisms for improving resilience under variable load [15]. Although this work does not focus on streaming RPC, its principles

are directly applicable to streaming workflows, where failures and backpressure must be managed continuously rather than per request.

Studies on high-volume data processing in microservices highlight the importance of explicit flow control and decoupling between producers and consumers. Guntupalli and Vamshi show that event-driven architectures combined with messaging systems can mitigate bottlenecks and stabilize performance under sustained data rates [5]. While these systems typically rely on brokers rather than direct streaming RPC, they reinforce the need for explicit backpressure mechanisms in streaming data paths.

From a systems perspective, Gan and Delimitrou demonstrate that communication overhead and tail latency dominate the performance of large-scale microservice deployments [4]. Their findings underline that resilience and scalability issues often emerge from inter-service communication rather than computation, an effect that is amplified in streaming scenarios.

Existing literature provides well-established principles for resilience and fault tolerance in microservice systems, but these are typically studied either in request-oriented architectures or broker-based streaming platforms. There is limited work examining how these mechanisms interact with direct streaming RPC in dynamically composed microservice chains, leaving questions about how failures and backpressure should be handled in dynamically composed microservice systems.

2.5 Validation practices, scalability, and performance evaluation

Empirical evaluation of streaming communication often focuses on comparing streaming and non-streaming interaction styles in terms of throughput, latency, and resource usage. Several benchmarking studies show that streaming RPCs outperform unary calls for large payloads and continuous data flows by enabling incremental processing and reducing buffering overhead [18]. Similar conclusions are drawn in broader comparisons between REST and gRPC, where binary protocols combined with streaming semantics demonstrate superior performance under load [11, 17].

Scalability in containerized streaming systems has been studied through systematic mapping and empirical evaluations. A mapping study on performance analysis techniques for streaming workloads in containerized microservices identifies network contention, scheduling overhead, and coordination costs as primary scalability bottlenecks [16]. Empirical studies of streaming pipelines deployed on Kubernetes further show that streaming approaches scale more gracefully than request-response models when concurrency increases, provided that backpressure and flow control are properly configured [19].

However, most existing evaluations assume static microservice topologies and fixed execution paths. Prior performance studies rarely consider adaptive middleware that dynamically constructs execution chains based on policy evaluation. As a result, there is limited empirical insight into how streaming communication scales when embedded within dynamically adaptive, policy-driven systems such as DYNAMOS. This lack of combined evaluation of streaming, scalability, and adaptivity motivates the experimental focus of this thesis.

2.6 Concept matrix

Work	Streaming tech	Broker vs RPC	Arch. integration	Policy & security	Resilience	Scalability & perf.
[20]	—	—	✓	✓	✓	✓
[21]	—	—	✓	✓	✓	✓
[19]	✓	—	—	—	✓	✓
[18]	✓	—	—	—	—	✓
[8]	—	✓	—	—	✓	✓
[14]	—	✓	—	—	✓	—
[1]	✓	✓	—	—	✓	✓
[2]	—	—	✓	—	✓	—
[13]	—	—	—	✓	—	—
[22]	—	—	—	✓	—	—
[10]	✓	✓	—	—	—	✓
[16]	✓	—	—	—	—	✓
[6]	—	—	—	—	—	✓
[9]	✓	—	—	—	✓	✓
[3]	✓	—	—	—	✓	✓
[24]	✓	—	—	—	✓	✓
[12]	✓	—	—	—	—	✓
[23]	✓	—	—	—	✓	✓

Table 1: Concept matrix of related work organised by thematic contribution.

2.7 Research gap

The literature review reveals that while individual aspects relevant to this thesis have been studied, several important gaps remain when these aspects are considered together.

2.7.1 Streaming communication and broker comparison

A substantial body of work addresses streaming technologies in distributed systems. Many studies focus on performance characteristics of stream processing frameworks such as Apache Spark, Flink, and Storm [3, 9, 12]. However, most research on comparing different communication approaches evaluates either traditional request-response patterns or compares streaming frameworks in isolation. When we look at work that explicitly compares brokers with RPC mechanisms, we find only a handful of studies [1, 8, 10, 14]. Of these, most focus on unary request-response communication rather than streaming semantics. Karimov et al. [9] provide a comprehensive benchmark of distributed stream processing systems, but their comparison is limited to Apache

Storm, Spark, and Flink. They do not consider gRPC streaming or compare RPC-based streaming with broker-based approaches. Similarly, Coviello et al. [1] introduce DataXc and compare it with gRPC streaming, but they evaluate their own solution rather than comparing gRPC streaming against established message brokers like Kafka or RabbitMQ. No existing work provides a direct empirical comparison between gRPC streaming and broker-based streaming that accounts for long-lived connections, flow control, and backpressure handling in realistic microservice deployments.

2.7.2 Architectural integration with policy-aware middleware.

The concept matrix shows that architectural integration and policy enforcement in streaming contexts are addressed by very few studies. This is partly because policy-aware, adaptive middleware like DYNAMOS represents a relatively new approach to microservice composition. While Stutterheim [20, 21] describes DYNAMOS and its policy-driven architecture. Some work touches on related aspects: Figueira and Coutinho [2] discuss self-adaptation in microservices, Neumaier et al. [13] explore policy-aware data exchange, and Thiyagarajan et al. [22] address security policies in gRPC contexts. However, none of these studies examine how streaming communication affects policy enforcement, service composition, or middleware behavior in an adaptive system. The question of how to integrate streaming semantics into a policy-driven middleware while preserving its adaptive capabilities remains largely unexplored.

2.7.3 Scalability and performance in streaming microservices.

Performance evaluation of streaming systems is well represented in the literature [9, 12, 16, 18, 24]. These studies typically measure throughput, latency, and resource utilization under various workloads. Research on scalability is less common but exists, with work by Henning and Hasselbring [6] proposing metrics for measuring scalability of stream processing engines, and Ververica [23] discussing scalability challenges in stream processing. What is missing is an evaluation of how streaming communication affects horizontal scalability in microservice architectures. Most performance studies focus on throughput and latency characteristics but do not systematically examine scaling behavior, particularly how systems respond to increasing numbers of service instances or growing workloads. The interaction between streaming semantics, service replication, load balancing, and adaptive middleware components has not been thoroughly investigated. Furthermore, existing scalability studies tend to focus on dedicated stream processing frameworks rather than on microservice-based systems where services dynamically compose into processing pipelines.

3 Methodology

This thesis adopts a mixed-methods approach combining systematic analysis, prototype implementation, and empirical evaluation to answer the research questions about streaming communication in DYNAMOS. Rather than immediately building a complete solution for one streaming technology, the research first establishes a solid understanding

of the available options and their trade-offs. This ensures the final integration choice is grounded in both theoretical analysis and practical experience.

The research unfolds in three phases, each addressing one of the sub-research questions. Each phase builds on the previous one, creating a clear path from initial exploration to validated implementation.

3.1 Streaming Feasibility of Data-Sharing Archetypes

Data-sharing platforms such as DYNAMOS are grounded in a set of recurring data-sharing archetypes that describe where data is located, where computation is executed, and which party controls the exchange under policy constraints [20, 21]. These archetypes, including Compute-to-Data and Data-through-a-Trusted-Third-Party (TTP), are typically described in terms of bounded request–response interactions, where a data-sharing request triggers a finite execution and produces a discrete result.

While DYNAMOS currently realizes these archetypes using atomic request–response communication, the archetype definitions themselves do not fundamentally restrict data exchange to batch-oriented or finite datasets. Rather, they specify control and trust boundaries, such as data immobility, centralized enforcement, or delegated execution, without prescribing whether data must be transferred or processed in its entirety before computation begins. This observation raises the question of whether these archetypes can be reinterpreted under a streaming communication model, where data is exchanged incrementally and computation proceeds continuously.

The Data-through-TTP archetype provides a particularly illustrative case. In this archetype, a trusted intermediary coordinates data exchange between multiple parties and enforces policy constraints on behalf of the participants [20]. Conceptually, this intermediary acts as a central routing and control point, a role that closely resembles the function of brokers or intermediaries in many streaming microservice architectures. Existing work on microservices-based streaming pipelines demonstrates that intermediary-mediated communication is a common and practical design pattern. For example, DataXc evaluates real-time streaming analytics pipelines in which all data exchange between microservices is routed through an intermediary communication layer, enabling flexible many-to-many communication while sustaining continuous data flows [1]. Although DataXc does not address policy-driven data sovereignty, it provides concrete evidence that centralized or intermediary-based streaming communication can operate efficiently in microservice environments.

Beyond specific implementations, the broader literature on streaming systems shows that continuous data processing with centralized coordination is well understood in terms of flow control, backpressure, and fault handling. These mechanisms are directly applicable to the Data-through-TTP archetype, where a trusted party can manage stream lifecycles, enforce access constraints over time, and aggregate or filter data incrementally rather than in a single batch. From this perspective, streaming does not invalidate the archetype’s assumptions but instead extends its execution model from short-lived jobs to long-lived, policy-governed sessions.

Although this thesis focuses primarily on a subset of archetypes implemented in DYNAMOS, the same reasoning can be extended to other data-sharing archetypes. Archetypes differ mainly in the placement of computation and control, not in the fundamental structure of data flow. As long as an archetype permits incremental processing and maintains clearly defined control boundaries, it can, in principle, be mapped onto a streaming communication model. Identifying where this mapping is straightforward and where it introduces new constraints is therefore a prerequisite for designing streaming support in DYNAMOS.

In summary, prior work on data-sharing archetypes establishes the structural patterns and trust assumptions that govern data exchange, while research on streaming microservice architectures demonstrates the feasibility of continuous, intermediary-mediated communication. However, existing literature does not explicitly connect these two perspectives. This thesis addresses this gap by examining how streaming communication can be aligned with data-sharing archetypes and integrated into a policy-driven, dynamically adaptive middleware such as DYNAMOS.

3.2 Phase 1: Survey and Selection of Streaming Technologies

Goal: Identify and compare streaming technologies suitable for large-scale data exchange in microservice architectures (RQ1).

The research begins with a broad survey of streaming technologies and frameworks used in distributed systems. This includes both academic literature on stream processing and industrial solutions deployed in production environments. The goal is not to find the perfect technology immediately, but to understand what options exist and what trade-offs they present.

A comparison framework will be created based on criteria that matter for DYNAMOS and streaming in general. Foundational evaluations of stream processing systems emphasize throughput, latency, fault tolerance, and scalability as core performance indicators for continuous data flows [6, 12]. These metrics have become standard in the benchmarking of stream processing systems and influence design considerations for streaming integrations in microservice architectures [23]. The comparison framework will include:

- **Performance characteristics:** How do different technologies handle throughput and latency? What are typical performance profiles under various workloads? Throughput is measured in records per second, while latency distributions characterize the system’s ability to sustain continuous flows without bottlenecks [7, 9].
- **Streaming semantics:** What delivery guarantees do they provide (at-least-once, exactly-once, at-most-once)? How do they handle message ordering? These communication semantics are critical for ensuring data consistency in distributed systems.

- **Operational behavior:** How do they manage backpressure when consumers cannot keep up? How do they handle dynamic workload changes? These operational characteristics determine practical deployability.
- **Resource requirements:** What are the CPU and memory costs? Are there specific infrastructure dependencies? Resource provisioning relates directly to load intensities and performance objectives [6].
- **Fault handling:** How do they behave when things go wrong? What recovery mechanisms exist? Fault tolerance and robustness under failure conditions are crucial for continuous operation [24].

This analysis phase deliberately avoids DYNAMOS-specific implementation concerns. The comparison is designed to be valuable not just for this thesis but for anyone researching streaming technologies in microservice contexts. This means other researchers can use the findings even if they are not working with DYNAMOS.

Starting with a technology-agnostic comparison serves two purposes. First, it prevents premature commitment to a solution that seems easy to implement but may have fundamental limitations. Second, it produces research findings that have broader applicability beyond just DYNAMOS. This two-stage selection, consisting of a general capabilities comparison followed by contextual feasibility assessment for DYNAMOS, is intended to produce results that are both academically relevant and practically actionable.

After this general comparison, the research will narrow down to a shortlist of 2–3 promising candidates by considering DYNAMOS-specific factors like compatibility with existing middleware components and architectural constraints.

The main challenge is that streaming technologies are designed for different use cases and make different assumptions. Comparing them fairly requires understanding not just their features but their underlying design philosophies. Additionally, vendor documentation and academic papers often emphasize different aspects, making direct comparison challenging. The research will systematically extract architectural properties, communication semantics, and operational behaviors from both academic surveys and industrial white papers to address this challenge.

3.3 Phase 2: Prototype Implementation and Integration Design

Goal: Design and implement streaming support in DYNAMOS while preserving its core architectural properties (RQ2).

For each technology shortlisted in Phase 1, a minimal working prototype integrated with DYNAMOS will be built. These prototypes are intentionally simple, they implement just enough functionality to reveal how the technology interacts with DYNAMOS’ architecture. This approach follows established practices in empirical software engineering for distributed systems research [3, 6].

Each prototype will demonstrate:

- How streaming channels are established between microservices in a dynamic composition
- How data flows through the middleware layer
- How DYNAMOS’ policy enforcement mechanisms interact with streaming connections
- How resource isolation works when services maintain persistent connections

The research takes an iterative approach: design the integration architecture first, then implement the minimal prototype, then evaluate what was learned. This cycle helps understand not just whether something works, but how much effort it takes and what compromises are needed.

For each prototype implementation, the following aspects will be systematically documented:

- **Integration complexity:** How many lines of code were needed? Which DYNAMOS components required modification? How many new dependencies were introduced? This quantitative assessment provides concrete measures of implementation effort.
- **Policy enforcement:** Can DYNAMOS still enforce policies on streaming data? Do policies need to be checked differently for streams versus request-response? Policy-driven orchestration is a core property of DYNAMOS that must be preserved.
- **Isolation guarantees:** Do persistent streaming connections compromise DYNAMOS’ isolation between jobs? Can proper resource boundaries be maintained? Isolation semantics are critical for multi-tenant deployments.
- **Fault handling:** What happens when a service in the stream fails? Can the system recover gracefully? How does it interact with DYNAMOS’ existing error handling? Robustness under failure conditions has been highlighted in recent studies of stream processing frameworks under fault injection [24].

The research deliberately avoids building complete, production-ready implementations. Full implementations would take too much time and might bias the research toward the first technology implemented. Instead, minimal prototypes allow quick exploration of multiple options and enable an informed decision about which approach to pursue further.

The prototype phase also serves another purpose: it reveals unexpected integration challenges that are not obvious from documentation. For example, a technology’s threading model might conflict with DYNAMOS’ concurrency assumptions, or its configuration system might not fit well with dynamic service composition.

The main challenge is reconciling persistent streaming connections with DYNAMOS’ ephemeral job execution model. DYNAMOS was designed around short-lived request-response interactions with clear boundaries. Streaming introduces long-lived connections that blur these boundaries. The research must ensure that adding streaming does not

undermine the isolation and policy enforcement that make DYNAMOS useful in the first place. Persistent sessions are common in streaming frameworks but can undermine isolation if not properly scoped within job lifecycles.

3.4 Phase 3: Performance Evaluation and Scalability Testing

Goal: Empirically validate the streaming integration through controlled experiments and assess its scalability (RQ3).

Systematic performance experiments will be conducted comparing the streaming-enabled DYNAMOS against the baseline request-response version. The experiments use controlled workloads in a Kubernetes environment where behavior can be precisely measured. Performance evaluation in streaming systems research traditionally focuses on throughput and latency, as these metrics characterize the system’s ability to sustain continuous flows without bottlenecks [7, 9].

The experimental design includes:

- **Workload variations:** Different data sizes (small messages versus large payloads), different concurrency levels (few versus many parallel streams), and different deployment scales (single node versus distributed cluster). Workloads will vary data sizes, concurrency levels, and deployment scale to cover representative scenarios.
- **Metrics collection:** Throughput (records per second), latency distributions (especially p95 and p99 percentiles), CPU and memory utilization, and network bandwidth usage. Increased emphasis is placed on tail latency percentiles because streaming workloads exhibit variability that average latency alone cannot capture [9].
- **Fault injection:** Deliberate introduction of failures (node crashes, network disruptions, service restarts) to test recovery behavior. Observing system behavior under fault conditions, such as node restarts or network disruptions, is critical for assessing robustness and aligns with prior evaluations of streaming frameworks [24].

Throughput and latency are standard metrics in streaming systems research, but the research is particularly interested in tail latencies (p95, p99) rather than just averages. Streaming workloads often show high variability; a few slow requests can significantly impact user experience even if the average latency looks good.

Fault injection is included because streaming systems need to handle failures gracefully. A system that performs well under ideal conditions but fails when something breaks is not useful in production. By deliberately causing failures, the research can measure how quickly the system recovers and whether it maintains consistency.

3.4.1 Scalability analysis

Beyond raw performance, the research examines how the system scales:

- **Horizontal scaling:** What happens when more service replicas are added? Does throughput increase linearly, or are there bottlenecks?
- **Vertical scaling:** Can the system handle higher data rates on the same infrastructure by optimizing resource usage?
- **Scale-out behavior:** How does performance change as deployments move from small to larger clusters?

Scalability is considered both in terms of horizontal scaling (additional service replicas) and increasing data rates. Scalability metrics draw on established literature that relates resource provisioning to load intensities and performance objectives [6]. This allows for defensible claims about how the integration performs as workloads grow, beyond anecdotal throughput measurements.

All streaming experiments include a baseline comparison with DYNAMOS' existing request-response mechanism. This shows not just that streaming works, but how much improvement it actually provides. The evaluation compares the streaming-enabled paths against the baseline request-response data transfers in DYNAMOS. For some workloads, streaming might not offer significant advantages. Understanding when streaming helps and when it does not is part of the contribution.

The main challenge is creating reproducible experiments. Distributed systems exhibit considerable variability, so enough repetitions are needed to achieve statistical confidence. The research must also ensure that the right things are being measured. It is easy to optimize for a metric that does not actually matter to real users.

3.5 Validation and Reproducibility

Throughout all phases, the research prioritizes reproducibility and transparency:

- **Controlled variables:** In performance experiments, everything except the communication mechanism being tested is controlled. Internal validity is supported by controlling variables other than the communication mechanism.
- **Synthetic workloads:** Deterministic workload generators are used rather than unpredictable real traffic, making experiments repeatable. Data will be synthetically generated to cover a range of stream volumes and message sizes, following approaches seen in prior benchmarking studies.
- **Clear metrics:** All measurements align with established practices in distributed systems research, making results comparable to prior work. Construct validity is ensured by aligning metrics with established research practices.
- **Documentation:** Implementation details, configuration parameters, and experimental procedures are thoroughly documented.

External validity is acknowledged in that certain findings may be specific to the DYNAMOS architecture and the selected streaming mechanisms, but the analytical analysis in Phase 1 promotes broader understanding of streaming trade-offs in microservice contexts.

3.6 Resource Requirements

The experiments require:

- A Kubernetes cluster capable of running DYNAMOS with multiple service replicas
- Synthetic workload generation tools for producing controlled streaming data
- Monitoring instrumentation including distributed tracing and metrics collection (such as distributed tracing systems)
- Storage for experimental data and logs

Synthetically generated data will be used following patterns seen in prior streaming benchmarks. The workload generator will be deterministic to support repeatability of results. This gives control over data characteristics while remaining realistic.

3.7 Timeline and Dependencies

The three phases are sequential but with feedback loops:

1. **Phase 1 output:** Shortlist of 2 to 3 candidate streaming technologies with documented trade-offs and a comprehensive understanding of the design space
2. **Phase 2 output:** Working prototypes demonstrating feasibility, with documented integration experiences including code complexity, system dependencies, and engineering challenges
3. **Phase 3 output:** Performance evaluation results and scalability analysis showing how the integration performs under various conditions

Each phase's results feed into the next. If Phase 2 reveals that a technology is much harder to integrate than Phase 1 suggested, the approach can be adjusted. If Phase 3 shows unexpected performance issues, iteration back to Phase 2 to investigate is possible. This iterative structure allows course-correction based on what is learned, rather than committing to a full implementation before understanding the trade-offs.

4 Risk Assessment

This project involves architectural design, prototype implementation, and empirical evaluation of streaming communication in a policy-driven, dynamically adaptive middleware. As such, it carries both technical and personal risks. This section identifies the most relevant risks, assesses their likelihood and impact, and defines concrete mitigation actions to reduce either the probability of occurrence or the severity of consequences.

Table 2 summarises the identified risks. Likelihood and impact are rated on a scale from 1 (very low) to 5 (very high). The risk score is computed as the product of likelihood and impact.

Description	Risk			Action
	Likelihood	Impact	Score	
Project goals are too ambitious given time constraints	3	5	15	Prioritise a minimal, well-defined streaming integration and restrict scope to one primary streaming approach; defer secondary optimisations if necessary
Project goals are too vague or insufficiently bounded	2	4	8	Formalise concrete success criteria early (supported streaming pattern, measurable performance metrics, comparison baseline) and validate them with the supervisor
Unexpected architectural complexity in DYNAMOS integration	3	5	15	Use minimal prototypes instead of full implementations; document integration obstacles early and adapt research focus to architectural analysis if needed
Unreliability or limitations of selected streaming technology	3	4	12	Shortlist multiple candidate technologies in Phase 1 and keep at least one fallback option that aligns closely with existing gRPC-based communication
Policy enforcement does not naturally extend to streaming communication	2	5	10	Explicitly design and document policy enforcement boundaries for streaming; if full enforcement proves infeasible, limit scope to clearly defined compliance guarantees
Performance evaluation results are inconclusive or noisy	3	3	9	Use controlled synthetic workloads, multiple repetitions, and focus on relative comparisons against the baseline rather than absolute performance claims
Gap between required distributed systems expertise and personal expertise	2	4	8	Allocate early time to studying DYNAMOS internals and relevant streaming frameworks; rely on incremental experimentation rather than large upfront changes
Supervisor or host contact temporarily unavailable (e.g. holidays)	2	3	6	Schedule regular progress meetings in advance and maintain written updates to allow asynchronous feedback during periods of unavailability
Loss of experimental data or configuration artifacts	1	5	5	Maintain version-controlled repositories and automated backups of experimental configurations, scripts, and results

Table 2: Risk register with likelihood, impact, and mitigation actions.

5 Ethics

This section evaluates potential ethical concerns and describes plans to address them. The main areas of ethical concern identified are evaluated below.

Human research participants and new data collection. This research does not involve human participants, surveys, interviews, or the collection of new data from people. All evaluation is conducted using synthetic workloads and controlled experimental data generated specifically for performance testing.

Existing datasets about people. The research does not use, analyse, or combine existing datasets containing personal information. Performance experiments rely entirely on artificially generated data with no relation to real individuals or organisations.

AI technologies and algorithms. While DYNAMOS is a system that could be used to orchestrate AI workloads, this thesis does not design, develop, or deploy AI technologies or machine learning algorithms. The focus is on communication mechanisms in distributed systems, not on AI-specific capabilities.

Cyber security and online privacy. The research addresses communication patterns in distributed systems, which has indirect relevance to security and privacy. However, the thesis does not involve penetration testing, vulnerability research, or deployment in production environments handling sensitive data. All experiments are conducted in isolated development and testing environments under the control of the CCI research group.

The streaming integration preserves DYNAMOS’s existing policy enforcement mechanisms, which are designed to maintain compliance with data-sharing agreements. While the thesis examines how policy enforcement interacts with streaming communication, it does not weaken or bypass existing security controls.

Dual-use technology. DYNAMOS is designed as infrastructure for policy-compliant data exchange in research and organisational contexts. The streaming extensions developed in this thesis are general-purpose communication improvements that enhance scalability and performance. While any communication infrastructure could theoretically be repurposed, the research does not create capabilities specifically designed for surveillance, intrusion, social manipulation, or military applications. The focus remains on improving data-sharing middleware for legitimate scientific and organisational use cases.

5.1 Ethical Safeguards and Compliance

Based on the above assessment, this research falls into a low-risk category with respect to the identified ethical concern areas. Nevertheless, the following safeguards will be maintained:

- All experimental deployments will use synthetic data with no relation to real individuals or organisations
- Performance testing will be conducted in isolated environments that do not interact with production systems or real user data
- Source code and architectural documentation will be reviewed to ensure no unintended vulnerabilities are introduced through the streaming integration
- Any deployment recommendations will explicitly address the need to maintain existing policy enforcement and security controls

5.2 Consultation and Approval

Should any unexpected ethical concerns arise during the research, for example, if collaboration with external organizations introduces access to real data, the Ethical Review Committee will be consulted immediately via the RMS portal for Faculty of Science.

No data collection, deployment in production environments, or experimentation with real user data will occur without prior ethical assessment and approval from the appropriate committees.

6 Project Plan

This section describes the planned execution of the thesis project, structured around clearly defined milestones and deliverables. The plan covers the entire scope of the project from initial orientation to final submission and is designed to make progress and risks observable at any point in time.

The project spans approximately 13 weeks, corresponding to the period from early April to the end of July. The timeline aligns with the three research phases defined in the methodology section: 1. Technology survey and selection; 2. Prototype implementation and architectural integration and; 3. Performance evaluation and validation. Writing is treated as a continuous activity with explicit intermediate deliverables to ensure steady progress.

6.1 Milestones and Deliverables

The following milestones define concrete, measurable achievements:

- **Milestone 1: Problem framing and baseline understanding (Week 2):** DYNAMOS architecture studied; baseline request–response communication path understood and documented; evaluation criteria for streaming technologies defined.
- **Milestone 2: Streaming technology shortlist (Week 4):** Literature study of streaming technologies completed; comparison framework applied; shortlist of 2–3 candidate technologies selected and justified in writing.

- **Milestone 3: Integration architecture design (Week 5):** Concrete architectural design for streaming integration in DYNAMOS produced, including decisions on communication patterns, policy enforcement boundaries, and lifecycle management.
- **Milestone 4: First working prototype (Week 7):** Minimal streaming prototype implemented and integrated with DYNAMOS, capable of executing a basic streaming data transfer within a dynamically composed microservice chain.
- **Milestone 5: Prototype refinement and documentation (Week 8):** Prototype extended with basic fault handling and instrumentation; integration complexity and architectural trade-offs documented.
- **Milestone 6: Performance evaluation completed (Week 11):** Controlled experiments executed; performance and scalability results collected for streaming and baseline implementations.
- **Milestone 7: First complete thesis draft (Week 10):** Full draft including introduction, related work, methodology, design, and preliminary evaluation results completed.
- **Milestone 8: Revised draft with evaluation results (Week 11):** Evaluation section finalized; discussion refined based on results and supervisor feedback.
- **Milestone 9: Final thesis submission (Week 12):** Final version completed, proofread, and submitted; all artifacts archived.

These milestones allow progress to be evaluated objectively at regular intervals and ensure that risks are detected early.

6.2 Timeline

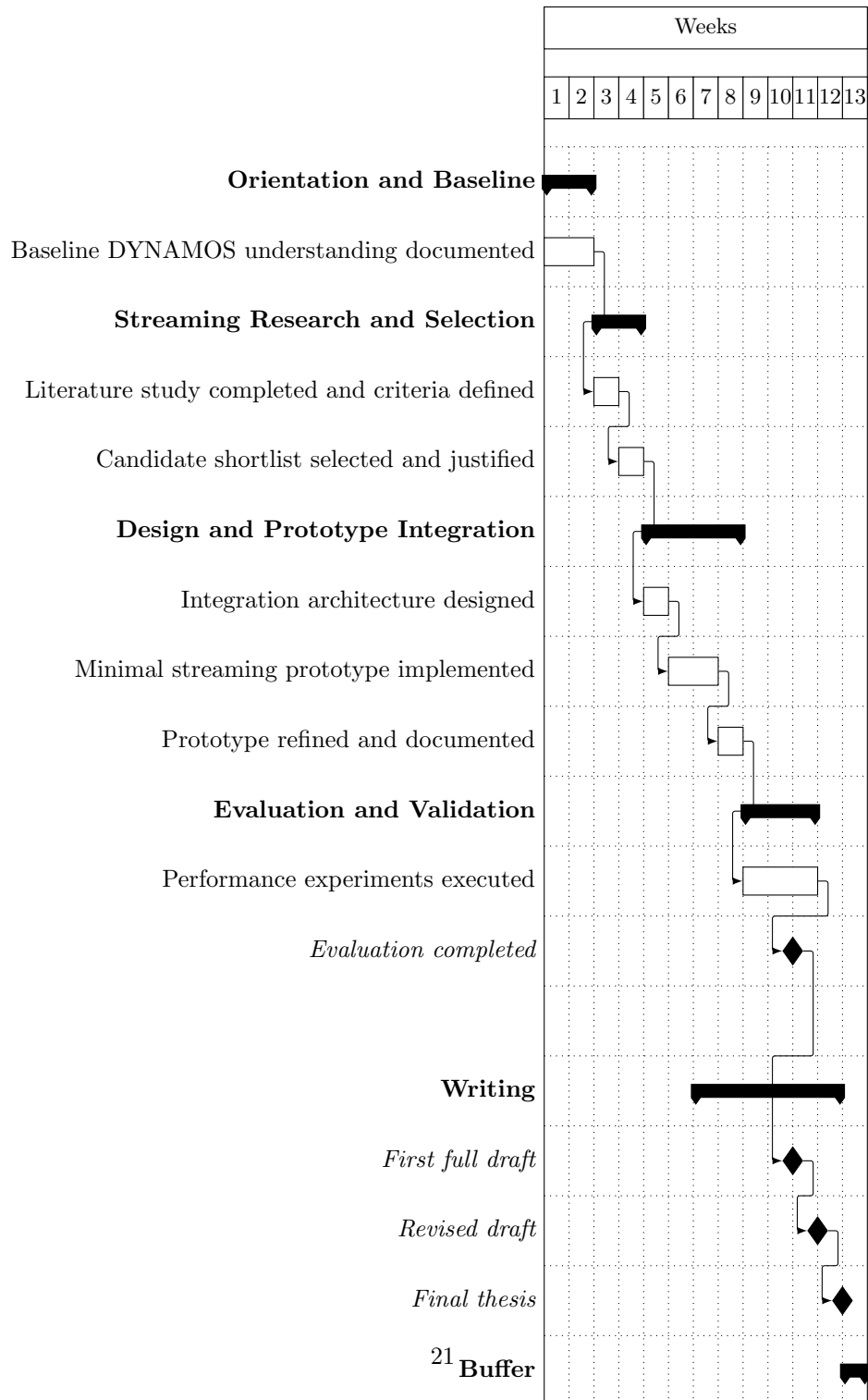


Figure 1: Project timeline with concrete milestones and deliverables.

6.3 Feasibility and Monitoring

The plan is deliberately conservative and feasible within a 3–5 month timeframe:

- Prototypes are explicitly scoped as *minimal*, avoiding the risk of overengineering.
- Writing overlaps with implementation and evaluation to reduce end-of-project pressure.
- A buffer week is reserved to absorb unexpected delays or revisions.
- Weekly progress can be assessed by checking milestone completion and deliverable status.

At any point in time, it is possible to determine whether the project is on schedule by comparing the current week against the planned milestone outcomes. This makes the plan suitable for continuous supervision and early intervention if deviations occur.

References

- [1] G. Coviello, K. Rao, C. G. D. Vita, G. Mellone and S. Chakradhar. “DataXc: Flexible and efficient communication in microservices-based stream analytics pipelines”. In: *2022 IEEE Intl Conf on Dependable, Autonomic and Secure Computing (DASC) / PiCom / CBDCom / CyberSciTech*. 2022. DOI: 10.1109/DASC/PiCom/CBDCom/Cy55231.2022.9927837.
- [2] J. Figueira and C. Coutinho. “Developing Self-Adaptive Microservices”. In: *Procedia Computer Science* 232 (2024), pp. 264–273. DOI: 10.1016/j.procs.2024.01.026.
- [3] M. Fragkoulis, P. Carbone, V. Kalavri and A. Katsifodimos. “A Survey on the Evolution of Stream Processing Systems”. In: *VLDB Journal* 33.2 (2024), pp. 507–541. DOI: 10.1007/s00778-023-00819-8.
- [4] Y. Gan and C. Delimitrou. “The Architectural Implications of Cloud Microservices”. In: *IEEE Computer Architecture Letters* 17.2 (2018), pp. 155–158. DOI: 10.1109/LCA.2018.2839189.
- [5] B. Guntupalli and S. Vamshi Ch. “Designing Microservices That Handle High-Volume Data Loads”. In: *International Journal of AI, Big Data, Computational and Management Studies* 4.4 (2023), pp. 76–87. DOI: 10.63282/3050-9416.IJAIBDCMS-V4I4P109.
- [6] S. Henning and W. Hasselbring. “How to Measure Scalability of Distributed Stream Processing Engines?” In: *Companion of the ACM/SPEC International Conference on Performance Engineering (ICPE ’21)*. ACM, 2021, pp. 85–88. DOI: 10.1145/3447545.3451190.

- [7] H. H. Hozhabr Pour, G. Ciortuz, A. Lüers and S. Fudickar. “Performance Analysis of a Data Stream Processing System for Online Activity Classification via Wearable Sensor Data”. In: *Proceedings of the 18th International Joint Conference on Biomedical Engineering Systems and Technologies (BIOSTEC 2025) - Volume 2: HEALTHINF*. SCITEPRESS – Science and Technology Publications, 2025, pp. 571–578. DOI: 10.5220/0013166100003911.
- [8] A. G. Ibrahim, R. P. Lopes, J. Rufino and P. Leitão. “On the Impact of Message Brokers Implementations in the Choreography of Microservices”. In: *Open Lecture Series 2A (OL2A 2025)*. Communications in Computer and Information Science (CCIS). Springer Nature Switzerland AG, 2026. DOI: 10.1007/978-3-032-00137-5_1.
- [9] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen and V. Markl. “Benchmarking Distributed Stream Data Processing Systems”. In: *Proceedings of the IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1519–1530. DOI: 10.1109/ICDE.2018.00169.
- [10] H. Kraft and R. Johansson. “Evaluating RPC for Cloud-Native 5G Mobile Network Applications”. Master’s thesis. Chalmers University of Technology and University of Gothenburg, 2020. URL: <https://hdl.handle.net/20.500.12380/302298>.
- [11] V. Lähtevänoja. “Communication Methods and Protocols Between Microservices on a Public Cloud Platform”. MA thesis. Espoo, Finland: Aalto University, School of Science, May 2021. URL: <https://urn.fi/URN:NBN:fi:aalto-202106207501>.
- [12] S. Lote, P. K B and D. Prater. “Real-time data stream processing in large-scale systems”. In: *World Journal of Advanced Research and Reviews* 15.3 (2022), pp. 560–570. DOI: 10.30574/wjarr.2022.15.3.0903.
- [13] S. Neumaier, G. Havur and T. Pellegrini. “Towards an Architecture for Policy-Aware Decentral Dataset Exchange”. In: *Proceedings of the Fifteenth International Conference on Advances in Semantic Processing (SEMAPRO 2021)*. Barcelona, Spain: IARIA, Oct. 2021. URL: https://www.researchgate.net/publication/358816711_Towards_an_Architecture_for_Policy-Aware-Decentral-Dataset-Exchange.
- [14] J. Nuikka. “Comparison of Cloud Native Messaging Technologies”. MA thesis. Tampere University, Faculty of Information Technology and Communication Sciences, May 2021. URL: <https://urn.fi/URN:NBN:fi:tuni-202104223314>.
- [15] J. A. Rasheedh and S. Saradha. “Reactive Microservices Architecture Using a Framework of Fault Tolerance Mechanisms”. In: *Proceedings of the Second International Conference on Electronics and Sustainable Communication Systems (ICESC)*. IEEE, 2021. DOI: 10.1109/ICESC51422.2021.9532893.
- [16] S. Ris, J. Araujo and D. Beserra. “A Systemic Mapping of Methods and Tools for Performance Analysis of Data Streaming with Containerized Microservices Architecture”. In: *2023 18th Iberian Conference on Information Systems and Technologies (CISTI)*. 2023, pp. 1–6. DOI: 10.23919/CISTI58278.2023.10211834.

- [17] Ritu, S. Arora, A. Bhardwaj, A. Kukkar and S. Kaur. “A Comparative Analysis of Communication Efficiency: REST vs. gRPC in Microservice-Based Ecosystems”. In: *Proceedings of the 2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)*. IEEE, 2024. DOI: 10.1109/INNOCOMP63224.2024.00107.
- [18] A. Sodankoor, A. Shenoy, B. M. Moger, M. Gowda, P. K. Auradkar and S. Kalambur. “Benchmarking gRPC Protocol on Various Virtualization Technologies”. In: *ICT4SD 2025*. Vol. 1654. Lecture Notes in Networks and Systems. Springer Nature Switzerland AG, 2026, pp. 316–329. DOI: 10.1007/978-3-032-06697-8_32.
- [19] C. Starck and J. Ghofrani. “Improving Load Balancing of Long-lived Streaming RPCs for gRPC-enabled Inter-service Communication”. In: *Proceedings of the 15th Central European Workshop on Services and their Composition (ZEUS 2023)*. Ed. by S. Böhm and D. Lübke. Vol. 3386. CEUR Workshop Proceedings. Hannover, Germany, 2023, pp. 45–51. URL: <https://ceur-ws.org/Vol-3386/>.
- [20] J. Stutterheim. “DYNAMOS: Dynamically Adaptive Microservice-based OS: A Middleware for Data Exchange Systems”. MA thesis. University of Amsterdam, 2023.
- [21] J. Stutterheim, A. Mifsud and A. Oprescu. “DYNAMOS: Dynamic Microservice Composition for Data-Exchange Systems, Lessons Learned”. In: *Proceedings of the IEEE International Conference on Software Architecture (ICSA)*. 2023.
- [22] G. Thiagarajan, V. Bist and P. Nayak. “Strengthening gRPC Security in Microservices: A Proxy-based Approach for mTLS, JWT, and RBAC Enforcement”. In: *International Journal of Computer Applications* 187.28 (Aug. 2025). DOI: 10.5120/ijca2025925359.
- [23] Ververica. *Streamouting Stream Processing Scalability: Challenges and Solutions*. <https://www.ververica.com/blog/stream-processing-scalability-challenges-and-solutions>. Accessed: 2026-01-16. July 2023.
- [24] A. Vogel, S. Henning, E. Perez-Wohlfeil, O. Ertl and R. Rabiser. “A Comprehensive Benchmarking Analysis of Fault Recovery in Stream Processing Frameworks”. In: *Proceedings of the 18th ACM International Conference on Distributed and Event-Based Systems (DEBS ’24)*. ACM, 2024, pp. 171–182. ISBN: 979-8-4007-0443-7. DOI: 10.1145/3629104.3666040.